TP GUIDÉ – Manipulation des vues et services transverses (Laravel)

Objectif général :

Découvrir et pratiquer la création de vues, la transmission de données, l'utilisation de layouts Blade, la génération d'URL, les sessions, la validation de formulaires, la gestion des erreurs et le logging dans Laravel.

Règles du TP

1. Suivre les étapes dans l'ordre.
2. Copier-coller **ou taper le code** dans Laravel.
3. Tester chaque route dans le navigateur.
4. Observer le résultat et lire les logs ou session si indiqué.
5. Noter les explications pour bien comprendre le rôle de chaque ligne.

Étape 1 – Première vue simple

Travail à faire :

1. Créer un fichier resources/views/product.blade.php avec le contenu :

```
<html>
<head>
  <title>Produit</title>
</head>
<body>
  <h1>Produit en vedette</h1>
  <p>Bienvenue dans notre boutique</p>
</body>
</html>
```

2. Créer une route pour afficher cette vue :

```
Route::get('/product', function () {
  return view('product');
```

```
});
```

À observer :

- Naviguer vers http://localhost/product
- Le texte “Produit en vedette” et “Bienvenue dans notre boutique” s’affiche

À retenir :

- view('product') charge le fichier Blade et Laravel l’envoie dans une réponse HTTP HTML.

Étape 2 – Vue avec données dynamiques

Travail à faire :

1. Modifier product.blade.php :

```
<h1>{{ $name }}</h1>
<p>Prix : {{ $price }} €</p>
```

2. Modifier la route pour transmettre les données :

```
Route::get('/product', function () {
   return view('product', [
      'name' => 'Ordinateur Portable',
      'price' => 799
   ]);
});
```

À observer :

- Affichage : “Ordinateur Portable” et “Prix : 799 €”

À retenir :

- Les variables $name et $price viennent du tableau passé à view()
- Blade utilise {{ }} pour sécuriser et afficher les variables

Étape 3 – Utilisation de ->with()

Travail à faire :

```
Route::get('/product-smartphone', function () {
   return view('product')
      ->with('name', 'Smartphone')
```

```
    ->with('price', 499);
});
```

À observer :

- Affichage : “Smartphone” et “Prix : 499 €”

À retenir :

- ->with() ajoute des variables à la vue
- Équivalent au tableau de l’étape 2

Étape 4 – Layout Blade (Template)

Travail à faire :

1. Créer le layout principal resources/views/layouts/app.blade.php :

```
<!DOCTYPE html>
<html>
<head>
  <title>E-Shop</title>
</head>
<body>
<header>
  <h1>Ma Boutique Laravel</h1>
</header>

<main>
  @yield('content')
</main>

<footer>
  <p>© 2026</p>
</footer>
</body>
</html>
```

2. Créer la vue resources/views/home.blade.php :

```
@extends('layouts.app')


@section('content')
   <h2>Accueil</h2>
   <p>Bienvenue dans notre boutique</p>
@endsection
```

3. Créer la route :

```
Route::get('/home', function () {
   return view('home');
});
```

À observer :

- Naviguer vers /home
- Le header, le contenu et le footer s'affichent

À retenir :

- @extends = hérite du layout
- @yield('content') = zone dynamique
- @section('content') = contenu injecté dans la zone

Étape 5 – Route nommée et génération d'URL

Travail à faire :

1. Créer une route nommée :

```
Route::get('/product', function () {
   return view('product', ['name' => 'Laptop', 'price' => 899]);
})->name('product.show');
```

2. Dans une vue (home.blade.php) ajouter :

```
<a href="{{ route('product.show') }}">Voir le produit</a>
```

À observer :

- Le lien génère automatiquement l'URL /product

À retenir :

- route('product.show') génère l'URL automatiquement même si l'URL change

Étape 6 – Sessions et message flash

Travail à faire :

1. Créer une route pour stocker une session :

```
Route::get('/set-session', function () {
   session(['user' => 'Ahmed']);
   return 'Session créée';
});
```

2. Créer une route pour afficher la session :

```
Route::get('/get-session', function () {
   return session('user');
});
```

3. Message flash après redirection :

```
Route::get('/add-success', function () {
   return redirect('/home')->with('success', 'Produit ajouté');
});
```

Dans home.blade.php afficher le message :

```
@if(session('success'))
   <div style="color:green">{{ session('success') }}</div>
@endif
```

À observer :

- Visiter /set-session puis /get-session → "Ahmed"
- Visiter /add-success → message "Produit ajouté" affiché une seule fois

Étape 7 – Validation d'un formulaire

Travail à faire :

1. Créer la vue resources/views/form.blade.php :

```
<form action="/submit" method="POST">
```

```
    @csrf
    Email: <input type="text" name="email"><br>
    Mot de passe: <input type="password" name="password"><br>
    <button type="submit">Envoyer</button>
</form>

@error('email')
    <p style="color:red">{{ $message }}</p>
@enderror

@error('password')
    <p style="color:red">{{ $message }}</p>
@enderror
```

2. Créer la route pour afficher le formulaire :

```
Route::get('/form', function () {
    return view('form');
});
```

3. Créer la route pour traiter le formulaire :

```
Route::post('/submit', function (\Illuminate\Http\Request $request) {
    $request->validate([
        'email' => 'required|email',
        'password' => 'required|min:6'
    ]);
    return 'Formulaire valide';
});
```

À observer :

- Saisie incorrecte → erreurs affichées sous le champ
- Saisie correcte → message “Formulaire valide”

Étape 8 – Gestion des erreurs

Travail à faire :

1. Créer la vue personnalisée resources/views/errors/404.blade.php :

```
<h1>Page introuvable</h1>
<p>Désolé, cette page n'existe pas.</p>
```

2. Créer une route pour tester abort(404) :

```
Route::get('/test-404', function () {
   abort(404);
});
```

À observer :

- Naviguer vers /test-404 → page 404 personnalisée affichée

Étape 9 – Journalisation (Logging)

Travail à faire :

```
Route::get('/test-log', function () {
   \Illuminate\Support\Facades\Log::info('Utilisateur connecté');
   \Illuminate\Support\Facades\Log::error('Erreur de paiement');
   return 'Logs enregistrés';
});
```

À observer :

- Visiter /test-log
- Vérifier storage/logs/laravel.log → messages ajoutés